Computational Photography Project Report: Mayank Hirani and Nikitha Sundaram
https://drive.google.com/drive/folders/1srzIn3tl3HzMkMxEitOGOH4n2Z9NtubB?usp=share_link

Motivation and Impact

We chose the topic of day-to-night image translation because it presents an intuitive, visually evaluable challenge with real-world applications in areas like photography enhancement, image editing, and augmented reality. The problem is difficult to solve manually due to the wide dynamic range and semantic variance between day and night scenes. Furthermore, solving this task has potential for improving models in domains such as autonomous driving and localization. Our goal was to explore how GAN architectures can learn semantic and appearance-level transformations between day and night domains, and how to improve accuracy by introducing noise and disruptions to the training data. The motivation comes from a paper focused on improving signal processing accuracy using a pipeline process to add noise and using various methods to first synthesize the night image from the day image before using a model to turn it into a night image.

Approach

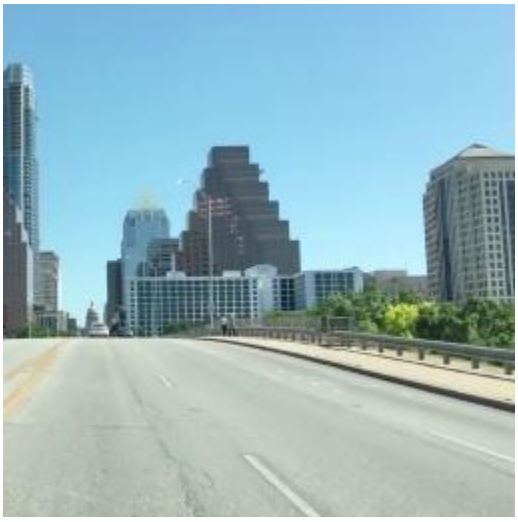
Our project involved the following stages:

1. **Import Dependencies and Setup:** We used Google Colab as our development environment and imported essential libraries such as PyTorch, torchvision, OpenCV, and PIL for deep learning and image processing tasks.
2. **Prepare Input Data:** We collected paired and unpaired day and night images from the Oxford RobotCar and BDD100K datasets. We resized all images to 256×256 resolution and normalized them for GAN training.
3. **Apply Preprocessing and Paper-Based Enhancements:** Inspired by techniques in key papers, we applied synthetic adjustments: we modified exposure, used gamma correction to change lighting, added night illumination effects using tints. Finally, we added Gaussian noise to the image.
4. **Train Standard pix2pixHD on Day Images:** We trained a pix2pixHD model using standard paired day-to-night and night-to-day image sets. We used adversarial loss along with reconstruction loss to guide the training.
5. **Train Noise-Augmented Model:** To improve robustness under poor conditions, we trained a variant of the model using noisy and damaged night images synthesized from the day images. This helped the generator better adapt to low-quality inputs.
6. **Prepare Test Set:** All test images were resized to 256×256 and duplicated across evaluation directories for comparison.

7. **Apply Standard pix2pixHD Model on Test Inputs:** We evaluated the standard model on the test set and observed how well it generalized to unseen data.
8. **Apply Noise-Adaptive pix2pixHD Model:** Finally, we applied the noise-adapted model to the same test set and compared the results to the baseline model to assess robustness and quality improvement.

Example of noise modifications added to a training day image:

Original



Modified

Results

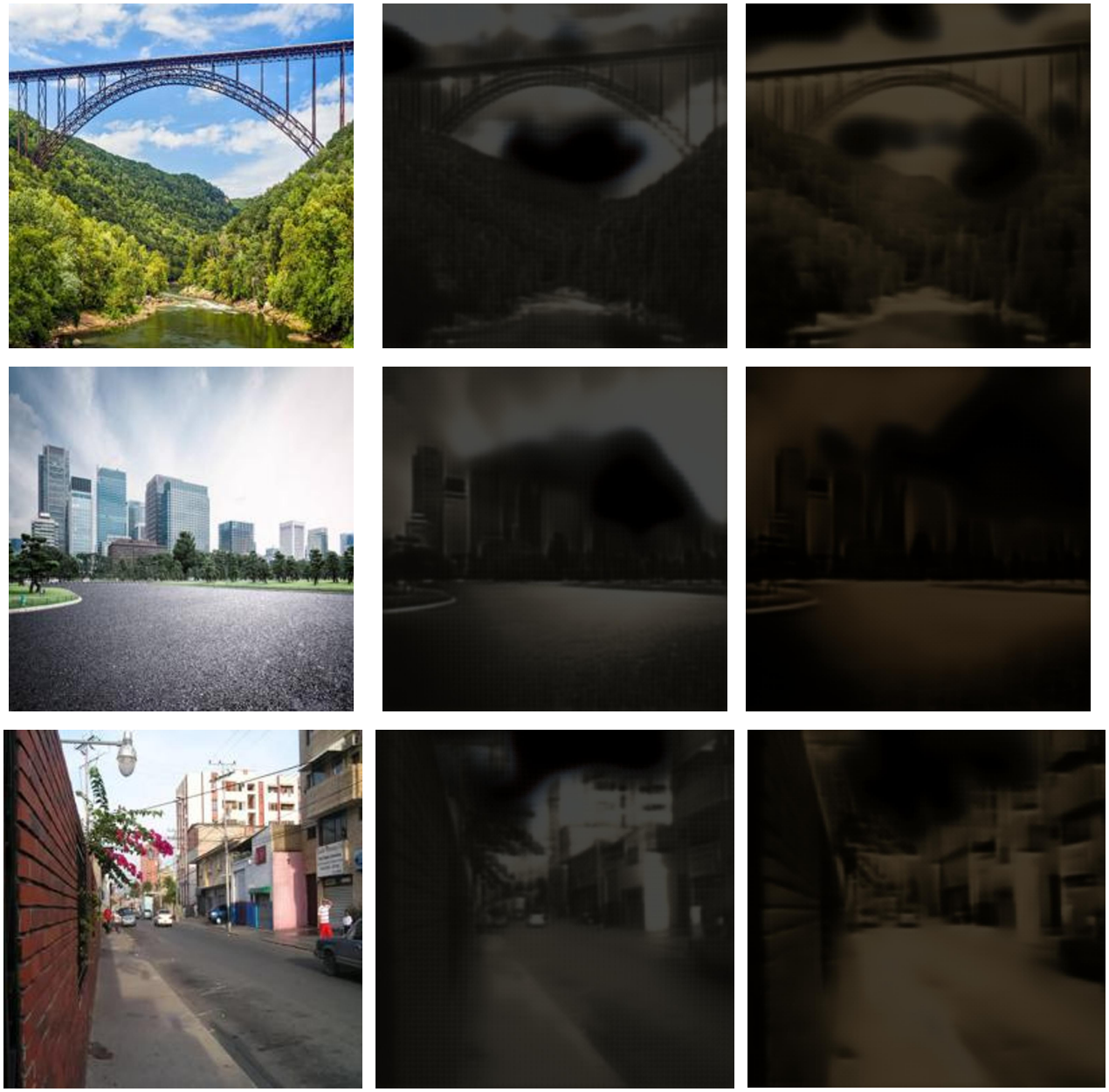
The standard pix2pixHD model produced visually convincing translations in clear scenes, with accurate color shifts (e.g., bright skies transforming into night skies with artificial lighting). However, it struggled with noisy or poorly lit inputs.

The noise-adapted model was designed to handle such inputs better by training on synthetically degraded images. While it occasionally preserved structure and context more effectively in adverse scenes, it did not consistently outperform the standard model. In fact, in some cases, the added noise may have introduced further degradation, making outputs worse. Unlike the referenced paper, our noise augmentation was simpler and less diverse, which likely limited the model's ability to generalize to various real-world noise conditions.

The standard model generally produced more coherent and higher-fidelity outputs on clean test images, while the noise-adapted model had mixed results—sometimes maintaining structure in noisy scenes but often struggling to improve over the baseline.

Image Results

Original day image, output of pix2pixHD standard, output of pix2pixHD on noisy training data



Implementation Details

- **Language & Framework:** Python, PyTorch, and TensorFlow
- **Computation:** Google Colab with GPUs
- **Libraries Used:** PyTorch Lightning, torchvision, NumPy, Matplotlib, OpenCV
- **Dataset:** Publicly available datasets used in referenced papers (e.g., Oxford RobotCar, BDD100K)
- **Code Structure:**
    - Data preparation to modify images and apply modifications and noise
    - Training code to train pix2pixHD on training data
    - Execution code to apply to test images

External code from GitHub was referenced for baseline CycleGAN and Pix2Pix models, with modifications to fit our dataset and experimental goals.

Challenge / Innovation

We faced one initial challenge with finding good training data. Lots of data online did not have the same image taken at both day and night, or had too small of a training dataset. Finally, we used the pedestrian training one, which had lots of images but with the drawback that they are all of streets or roads.

Another challenge we faced was with our initial approach, using stable diffusion and ControlNet to generate a night image that fit the edge map of the day image, and then apply post-processing modifications to make it better. However, this method resulted in lots of issues regarding memory and conflicting versions, so we decided to use pix2pixHD instead.

With our approach, we used a simpler method of synthesizing night images compared to the paper, and surprisingly achieved very good synthesized night image results. We also chose to use pix2pixHD to test the paper's methods using a different model.

We believe this project meets the criteria for the 20/20 challenge/innovation score due to the following reasons:

- We implemented a moderately complex vision task involving GANs with a nontrivial dataset and training regimen.
- We dealt with unclear steps from academic literature, adapted baseline models, and showed successful qualitative results.
- We wanted to extend to diffusion models if time had allowed, further increasing complexity and innovation, and possibly trying the stable diffusion and ControlNet method.